Università degli Studi di Urbino Carlo Bo

Corso di Laurea in Informatica – Scienza e Tecnologia

Programmazione e Modellazione a Oggetti, a.a. 2025/2026

Relazione per il Progetto del Corso di Programmazione e Modellazione a Oggetti

---

Mynimals

Tonino Zulli

Stefano Roberto Rusu

Agyiri Godfred Yeboah

Indice

1 Analisi **2**
- 1.1 Requisiti . . . 2
- 1.2 Modello del dominio . . . 3

2 Design **6**
- 2.1 Architettura . . . 6
- 2.2 Design dettagliato . . . 7
  - 2.2.1 Tonino Zulli — Mondo e movimento . . . 7
  - 2.2.2 Agyiri Godfred Yeboah — Battaglia e incontri . . . 11
  - 2.2.3 Stefano Roberto Rusu — Grafica e orchestrazione . . . 16

3 Sviluppo **23**
- 3.1 Testing automatizzato . . . 23
  - 3.1.1 La suite . . . 23
  - 3.1.2 Come verificare che le prove servano davvero . . . 24
  - 3.1.3 Come si eseguono . . . 24
- 3.2 Metodologia di lavoro . . . 24
  - 3.2.1 Tonino Zulli . . . 25
  - 3.2.2 Agyiri Godfred Yeboah . . . 25
  - 3.2.3 Stefano Roberto Rusu . . . 26
  - 3.2.4 Strumenti . . . 26
- 3.3 Note di sviluppo . . . 26
  - 3.3.1 Tonino Zulli . . . 26
  - 3.3.2 Agyiri Godfred Yeboah . . . 27
  - 3.3.3 Stefano Roberto Rusu . . . 27
  - 3.3.4 Nota comune . . . 28

4 Commenti finali **29**
- 4.1 Autovalutazione e lavori futuri . . . 29
- 4.2 Difficoltà incontrate . . . 30
- 4.3 Commenti per i docenti . . . 30

A Guida utente **31**
- A.1 Requisiti . . . 31
- A.2 Come avviare il gioco . . . 31
- A.3 Comandi . . . 31
- A.4 Come si gioca . . . 32
- A.5 Struttura della consegna . . . 32

Capitolo 1

Analisi

Il progetto, chiamato *Mynimals*, consiste nello sviluppo di un gioco di ruolo 2D a visuale dall'alto, ispirato a Pokémon. Il giocatore si muove su una griglia di caselle, esplorando un mondo diviso in zone esterne collegate da passaggi; camminando nell'erba alta può imbattersi in Pokémon selvatici, e ogni incontro apre una battaglia a turni in cui si scelgono le mosse del proprio Pokémon oppure si tenta la fuga. I Pokémon appartengono a elementi diversi, con un sistema di efficacia reciproca che premia le scelte tattiche, e un edificio particolare — l'ospedale — permette di curare il proprio Pokémon.

In coerenza con le finalità del progetto d'esame, i Pokémon, i loro nomi, gli sprite e l'ambientazione sono interamente originali: l'ispirazione riguarda il genere e le meccaniche di gioco, non i contenuti, che sono stati ideati e disegnati da noi.

1.1 Requisiti

Requisiti funzionali

- Il giocatore si muove su una griglia di caselle nelle quattro direzioni cardinali, con un movimento animato in modo fluido da una casella all'altra.
- Il mondo di gioco è diviso in zone esterne collegate da passaggi: attraversando un passaggio il giocatore passa da una zona all'altra.
- Alcune caselle non sono percorribili (alberi, acqua, edifici) e bloccano il movimento; quando il passo è impossibile il personaggio si gira comunque verso l'ostacolo.
- Camminando nell'erba alta si possono generare incontri casuali con Pokémon selvatici.
- Un incontro apre una battaglia a turni fra il Pokémon del giocatore e quello selvatico.
- In battaglia il giocatore, al proprio turno, può usare una delle mosse del proprio Pokémon oppure tentare la fuga; l'incontro termina con vittoria, sconfitta o fuga riuscita.
- I Pokémon appartengono a elementi diversi, con un sistema di efficacia reciproca che rende alcune mosse più o meno efficaci a seconda dell'elemento del bersaglio.
- Alcuni edifici hanno una porta; la porta dell'ospedale cura il Pokémon del giocatore dopo una breve attesa. Gli interni degli edifici sono fuori dallo scope del progetto.
- In caso di sconfitta il giocatore viene riportato al proprio punto di partenza con il Pokémon curato, in modo che una battaglia persa non blocchi la partita.

Requisiti non funzionali

- L'applicazione deve essere portabile e compilabile ovunque: per eseguire il gioco deve bastare la libreria standard del linguaggio, senza dipendenze esterne da scaricare né risorse da procurarsi a parte.

- L'aspetto grafico deve richiamare i giochi "a pixel" degli anni '90, con pixel netti e senza sfocature, e la finestra deve essere ridimensionabile senza deformare la grafica.
- Il gioco deve mantenere una velocità di simulazione costante e indipendente dalla potenza della macchina e dal numero di fotogrammi al secondo.
- I Pokémon, i loro nomi, gli sprite e l'ambientazione devono essere originali, sia per motivi di copyright sia perché si tratta di un progetto d'esame.
- Identificatori e commenti del codice sono in italiano.

L'unico attore del sistema è il Giocatore. Il diagramma dei casi d'uso riassume le funzionalità a sua disposizione.

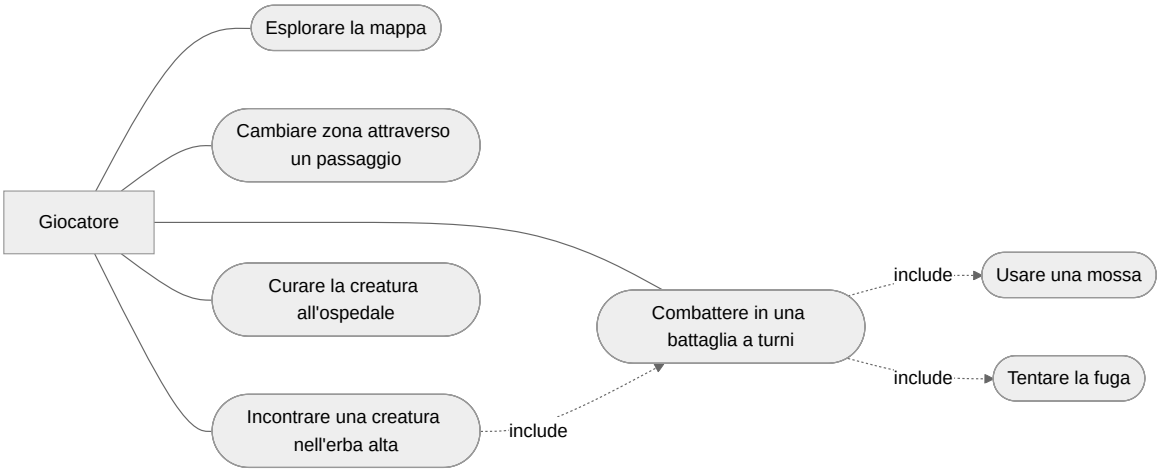


Figura 1.1: Diagramma dei casi d'uso: l'unico attore è il Giocatore.

1.2 Modello del dominio

Il dominio del gioco ruota attorno a un Mondo esplorabile e a un sistema di battaglia fra Pokémon. Di seguito si descrivono le entità del problema e le relazioni fra loro, senza riferimenti alla realizzazione.

Il Mondo è composto da più Zone. Ogni Zona è una griglia di Caselle, ognuna di un certo TipoTile (sentiero, erba, erba alta, albero, acqua, edificio, porta, ospedale) che ne determina due proprietà rilevanti per il dominio: se la casella è calpestabile e se può generare incontri. Alcune caselle sono Passaggi, che collegano una Zona a un'altra.

Il Giocatore occupa una Posizione (una casella della griglia) e combatte con un Pokémon. Un Pokémon appartiene a una Specie, che ne fissa le statistiche base (punti salute, attacco, difesa, velocità) e le Mosse che impara; a partire dalla specie e dal proprio livello il Pokémon ricava i valori effettivi e i punti salute correnti. Ogni Mossa ha un Elemento, una potenza e una precisione, ed è l'Elemento a stabilire quanto una mossa sia efficace contro il bersaglio.

Gli incontri sono descritti, per ogni Zona, da una TabellaIncontri composta da più VoceIncontro: ciascuna indica quale Specie può comparire e in quale intervallo di livelli. Camminando nell'erba alta, la Zona può generare un incontro pescando dalla propria tabella.

La Battaglia oppone il Pokémon del giocatore a un Pokémon selvatico e si conclude con un Esito: vittoria, sconfitta o fuga.

Nel codice un Pokémon è realizzato dalla classe `Creatura`: nei diagrammi e nelle descrizioni di progetto del Capitolo 2 si userà quindi il nome di classe `Creatura`, mentre in questo modello

del dominio si parla di Pokémon.

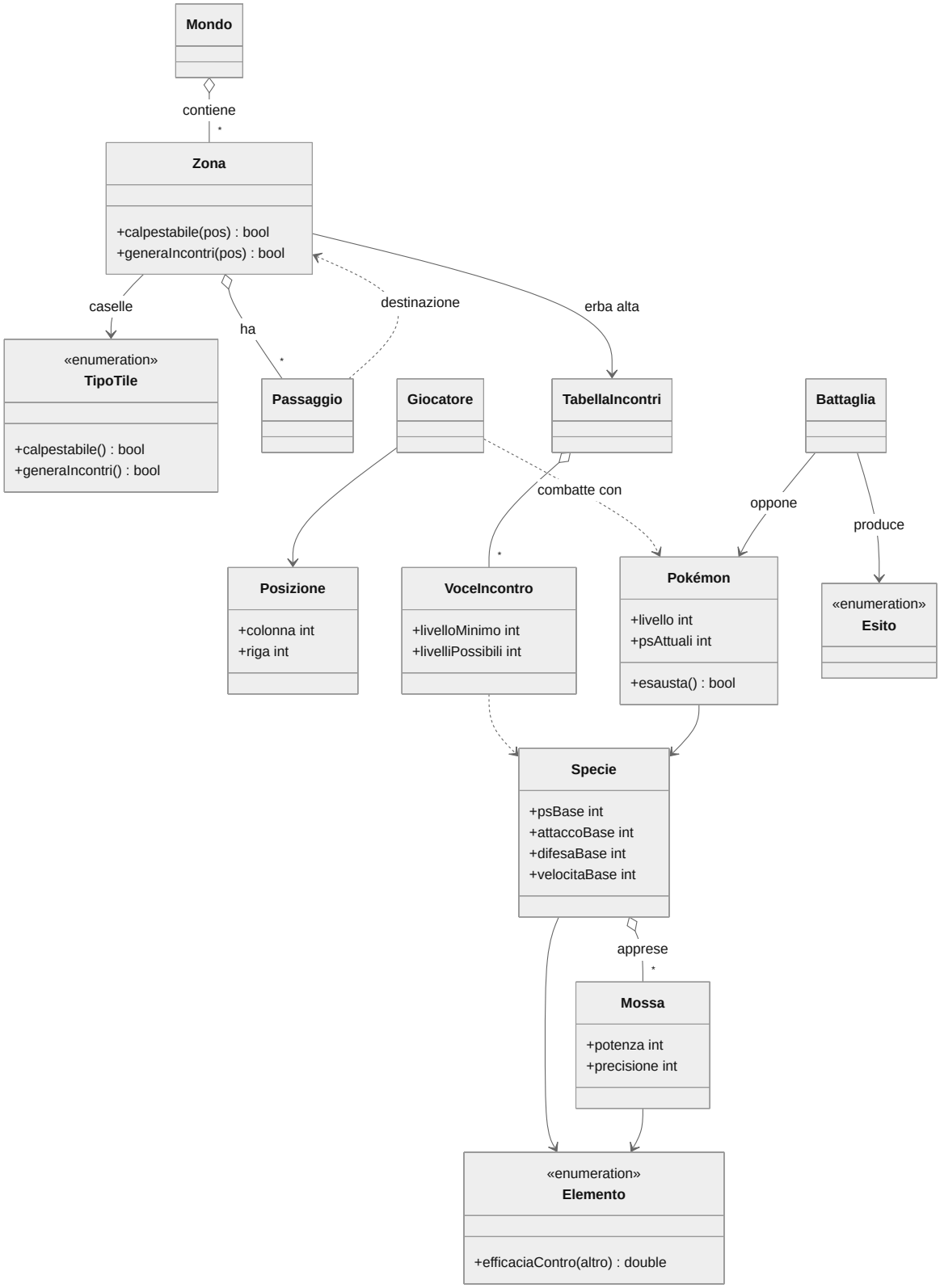


Figura 1.2: Schema UML del dominio: le entità principali del problema e le relazioni fra loro.

Capitolo 2

Design

2.1 Architettura

L'applicazione è strutturata secondo il pattern architetturale MVC (Model-View-Controller), con un'avvertenza che dichiariamo subito: il controller non è una classe sola. Il progetto è un GDR 2D a visuale dall'alto scritto in Java 21 e Swing: 50 file e circa 4.155 righe in 8 package nel sorgente, più 11 classi di test in `test/`.

Model — punto d'ingresso `gdr.partita.Partita`. Tiene lo stato (mondo, giocatore, Pokémon, fase di gioco), applica le regole e riferisce cosa è successo. Sotto di esso: `gdr.mondo` (la geografia: zone, passaggi, tipi di casella), `gdr.battaglia` (la macchina a turni), `gdr.incontri` (la regola degli incontri casuali), `gdr.entita` (il movimento a griglia), `gdr.nucleo` (i tipi base: posizione, direzione, sorgente casuale), `gdr.dati` (i contenuti: mappe, Pokémon, mosse). Non conosce Swing, non legge tasti, non disegna.

Una precisazione rispetto al dominio, utile per leggere il resto: nel dominio il giocatore combatte con un Pokémon, ma nel modello il Pokémon è tenuto da `Partita` e non da `Giocatore`, perché appartiene alla partita in corso e non al personaggio. `Giocatore` possiede soltanto il proprio movimento.

View — punti d'ingresso `RenderizzatoreEsplorazione` e `RenderizzatoreBattaglia`, in `gdr.grafica`. Leggono lo stato e producono pixel su una tela logica di 240x176, poi ingrandita di un fattore intero. Non modificano mai il Model.

Controller — punto d'ingresso `PannelloGioco`, assistito da `GestoreInput` (traduce i tasti in comandi) e `CicloDiGioco` (passo fisso di 1/60 di secondo).

Come comunicano. Dal Controller al Model entrano solo un `Optional<Direzione>`, un numero di secondi e un'`AzioneBattaglia`: il Model non vede mai un `KeyEvent`. Dal Model esce una lista immutabile di eventi — `EventoPartita`, `EventoEsplorazione`, `EventoBattaglia` — definiti come `sealed interface` di `record`. Il Model non chiama mai la View. Gli `switch` che consumano quegli eventi non hanno `default`: aggiungendone uno nuovo, il progetto smette di compilare finché la View non lo gestisce.

Dipendenza a senso unico. `gdr.grafica` importa i package di logica; nessun package di logica importa `gdr.grafica`, e nessun package di logica importa AWT o Swing. Sono due invarianti distinti e verificabili con due comandi, che non devono stampare nulla:

```
grep -rl "import gdr.grafica" src/ | grep -v gdr/grafica
grep -rl "import java.awt\|import javax.swing" src/ | grep -v gdr/grafica
```

Il Model sa cosa succede, il Controller sa quando. `Battaglia` risolve il turno intero all'istante e restituisce gli eventi già risolti; quanto resta in scena ciascun evento lo decide `AnimatoreBattaglia`, che sta nella View. E `chiudiBattaglia()` la chiama la View, quando l'animazione è finita: il Model sa che una sconfitta riporta al punto di partenza, non sa quando mostrarlo.

Limite noto, dichiarato. `PannelloGioco` fa due mestieri, superficie di disegno e controller, perché in Swing il componente che dipinge è anche quello che possiede la `InputMap`. Sostituendo in blocco la View, il Model non cambierebbe di una riga — l'intero gioco gira senza aprire finestre, ed è così che è stato collaudato — mentre il Controller andrebbe in parte riscritto. Si chiuderebbe estraendo un `ControlloreGioco` che possiede `Partita`, `GestoreInput` e `AnimatoreBattaglia`, lasciando a `PannelloGioco` solo il `paintComponent`.

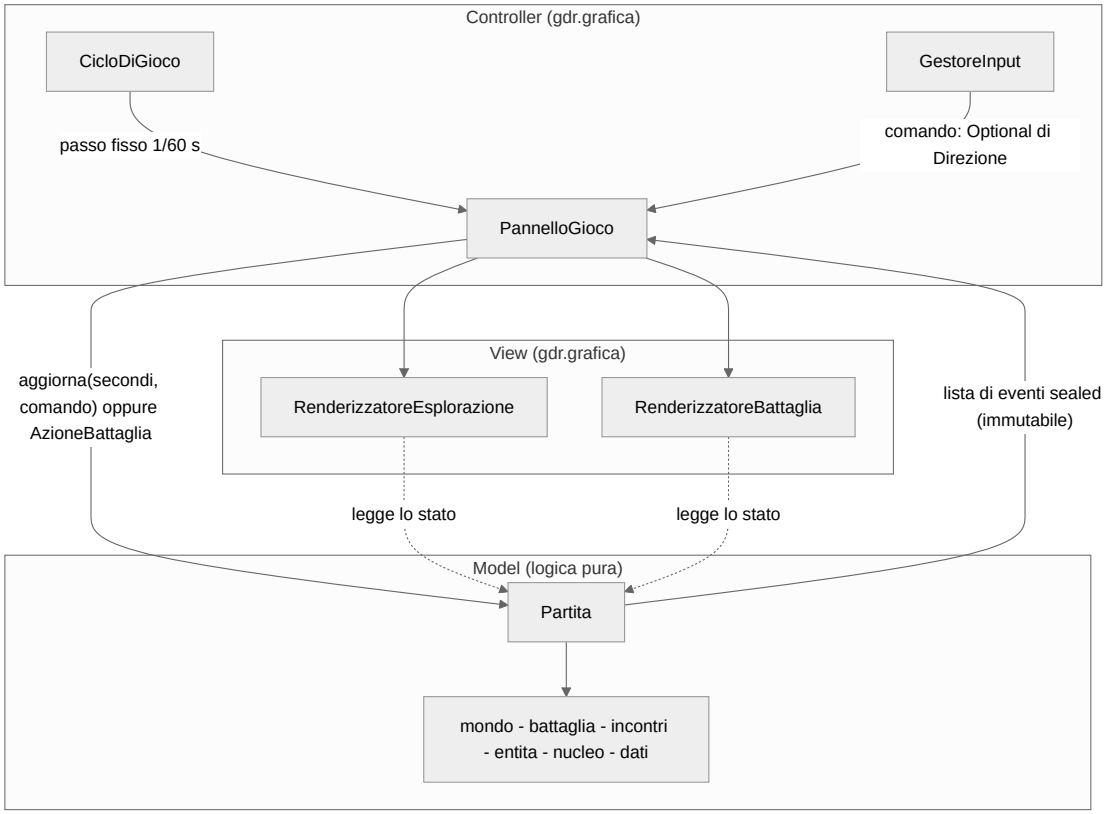


Figura 2.1: L'architettura MVC: comandi e azioni verso il Model, eventi immutabili verso la View. `gdr.grafica` dipende dalla logica, mai il contrario.

2.2 Design dettagliato

2.2.1 Tonino Zulli — Mondo e movimento

Mi sono occupato del mondo di gioco e del movimento del personaggio: i package `gdr.nucleo`, `gdr.mondo`, `gdr.entita`, la logica di esplorazione (`gdr.partita.Esplorazione` con i relativi eventi) e le mappe testuali in `gdr.dati.ZoneIniziali`. Il filo conduttore è tenere la logica indipendente dalla grafica e far emergere subito gli errori.

2.2.1.1 Il movimento a griglia con animazione fluida

Problema. Il personaggio deve spostarsi di casella in casella con un'animazione fluida, ma la posizione "vera" usata dalle regole (questa casella è erba alta? c'è un passaggio? è calpestabile?) deve restare non ambigua.

Soluzione proposta. `MovimentoGriglia` tiene la posizione logica sempre come casella intera, più un progresso fra 0 e 1 che serve soltanto a interpolare il disegno. Durante un passo la casella occupata resta quella di partenza; diventa quella di arrivo solo quando il passo è concluso. Chi disegna chiede `colonnaInterpolata()` e `rigaInterpolata()`, la logica chiede `casella()`. Un punto chiave: questa classe non conosce la mappa, quindi non sa cosa sia un muro; decidere se un passo è lecito spetta a `Esplorazione`, che ha la zona sottomano. Così il movimento resta verificabile da solo.

Alternative considerate. Tenere la posizione in coordinate continue (`double`) sarebbe stato più corto, ma avrebbe trasformato ogni regola del gioco in un confronto con una soglia e in un arrotondamento, oltre a introdurre micro-scatti: scartata. Inoltre, la prima versione riversava il tempo in eccesso nel passo successivo (camminata "continua"); è stata sostituita dalla regola *un aggiornamento conclude al massimo un passo*, perché quella continuità causava due difetti reali: un passo risolto con la mappa della zona sbagliata subito dopo un cambio zona, e un passo che sopravviveva alla battaglia facendo camminare il personaggio da solo. Con il passo fisso di 1/60 s e 4 caselle al secondo una casella dura esattamente 15 aggiornamenti, quindi non si perde fluidità.

Pattern applicati. Separazione delle responsabilità: il movimento conosce il proprio stato ma non le regole del mondo, e la decisione di liceità sta in un collaboratore che possiede la mappa. Oggetto immutabile: lo spostamento passa sempre per una nuova `Posizione`, mai per la modifica di una esistente.

2.2.1.2 Il mondo come dato, validato una volta sola

Problema. Le mappe sono scritte come testo. Un errore di battitura — un passaggio che punta a una zona inesistente, una casella d'arrivo dentro un muro — non deve diventare un vicolo cieco scoperto giocando mezz'ora dopo.

Soluzione proposta. Le mappe sono liste di stringhe; `LettoreZona` le trasforma in una `Zona` già rettangolare e piena, segnalando righe di lunghezza diversa o simboli sconosciuti. Il costruttore di `Mondo` verifica gli invarianti una sola volta, all'avvio, e solleva un'eccezione con un messaggio esplicito se qualcosa non torna: ogni passaggio punta a una zona esistente; la casella d'arrivo è dentro la zona di destinazione ed è calpestabile; la casella d'arrivo non è a sua volta un passaggio (niente rimbalzi); la zona di destinazione ha un passaggio che riporta indietro. Un errore di battitura diventa così un messaggio all'avvio invece di un comportamento strano scoperto giocando.

Alternative considerate. Un formato su file esterno (JSON, properties) sarebbe più "pulito" in astratto, ma aggiungerebbe un parser, la gestione degli errori di lettura e un file che può mancare a chi scarica il progetto, in cambio di nessun beneficio reale: le mappe le modifichiamo noi, non i giocatori. Scartata.

Trappola documentata. `Zona.tipo(pos)` lancia su una posizione fuori mappa, mentre `calpestabile(pos)` e `generaIncontri(pos)` restituiscono `false` senza esplodere: queste ultime rispondono a domande sensate anche fuori dai bordi ("posso andare lì?" — no), la prima no. Chi chiama `tipo()` su una casella incerta la protegge prima con `dentro()`.

Pattern applicati. Metodo Factory statico: `LettoreZona` ha costruttore privato ed è una

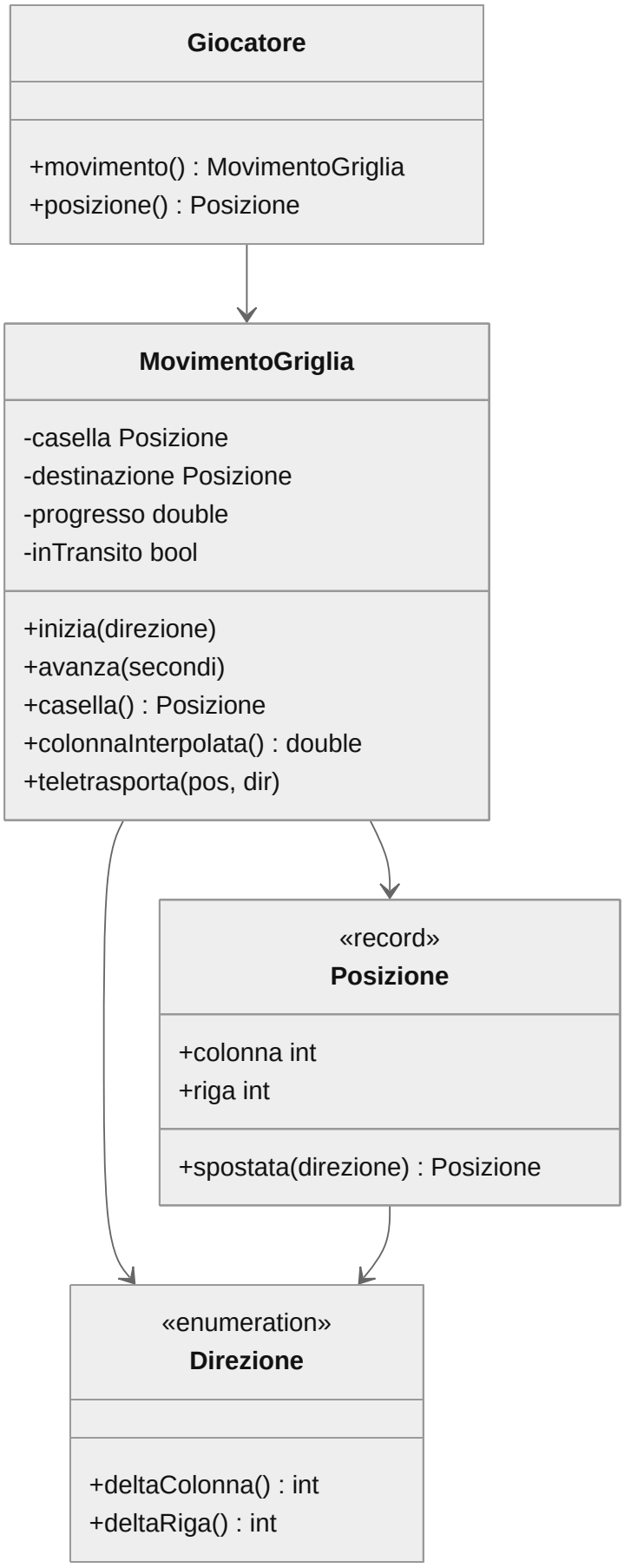


Figura 2.2: Il movimento e i tipi base del nucleo.

raccolta di funzioni che costruiscono una `Zona` valida, l'unico modo per crearne una. Oggetto immutabile / Value Object: `Posizione` è un `record`, e `spostata()` non modifica nulla ma restituisce una nuova posizione; `equals`/`hashCode` arrivano gratis e servono davvero (`Zona.passaggioIn` confronta posizioni).

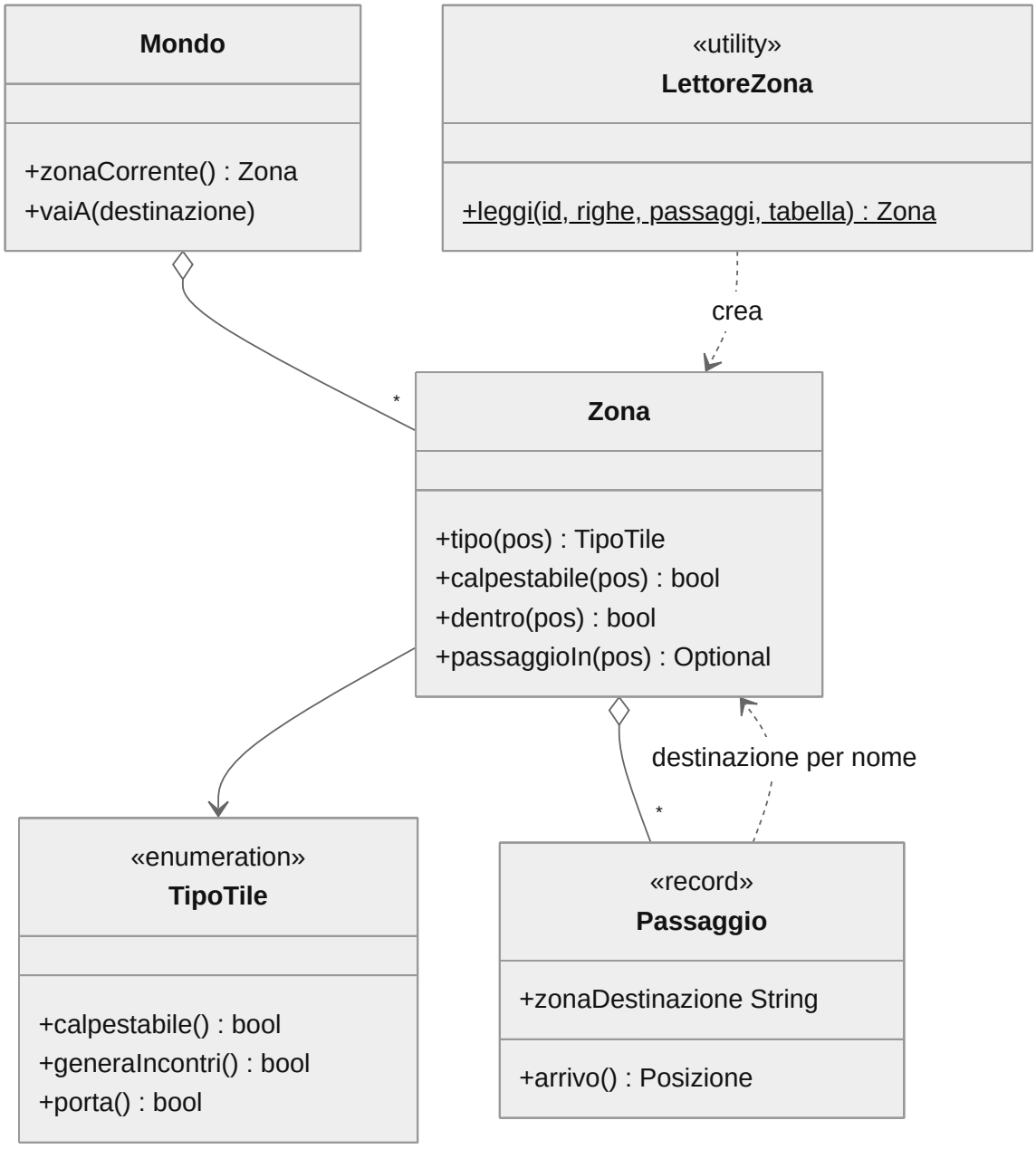


Figura 2.3: Il mondo: zone, caselle, passaggi e la loro costruzione validata.

2.2.1.3 Il confine fra logica e grafica

Problema. La logica di esplorazione non deve conoscere Swing, suoni o testo, ma deve comunque comunicare a chi disegna cosa è accaduto a ogni passo.

Soluzione proposta. `Esplorazione.aggiorna(secondi, comando)` riceve un `Optional<Direzione>` (vuoto se nessun tasto) e restituisce una lista immutabile di `EventoEsplorazione`: passo completato, porta toccata, zona cambiata. `Partita` legge quegli eventi e li traduce per la

vista; la logica non emette suoni, non scrive testo, non disegna. Un dettaglio che il codice rende esplicito: il passaggio si controlla *prima* di annunciare il passo, perché la casella che innesca un cambio zona non è una casella su cui si cammina, e annunciarla farebbe cercare un incontro nella zona appena raggiunta usando la casella di quella vecchia.

Alternative considerate. Passare alla logica un riferimento alla vista, o farle esporre dei listener da registrare, avrebbe ottenuto lo stesso risultato apparente ma con una dipendenza in più e senza la garanzia del compilatore: un evento nuovo si sarebbe potuto dimenticare in silenzio. Restituire una lista di fatti costa una allocazione per aggiornamento e in cambio rende la logica eseguibile, e collaudabile, da sola. Scartata anche l'idea di restituire un singolo evento invece di una lista: in un aggiornamento possono succedere due cose (un passo e una porta toccata).

Pattern applicato. Comunicazione a eventi con tipi `sealed`: `EventoEsplorazione` è una `sealed interface` di `record`, quindi chi la consuma usa uno `switch` esaustivo senza `default` e un evento nuovo non può essere dimenticato — il compilatore lo impone. La logica "parla, non comanda": restituisce fatti, non chiama la vista.

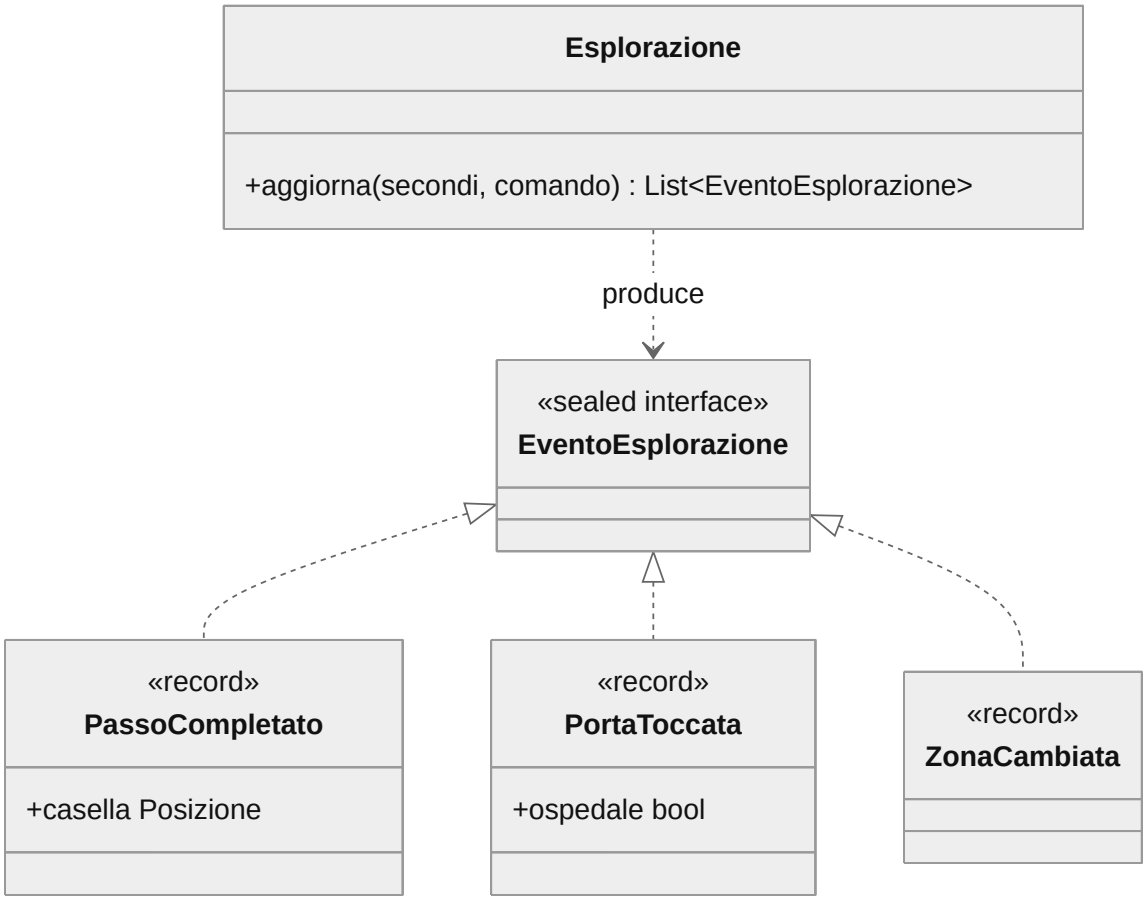


Figura 2.4: Gli eventi dell'esplorazione come gerarchia sealed.

2.2.2 Agyiri Godfred Yeboah — Battaglia e incontri

Mi sono occupato del sistema di battaglia e degli incontri: i package `gdr.battaglia`, `gdr.incontri` e i contenuti di gioco in `gdr.dati.Bestiario` e `gdr.dati.Repertorio`. L'obiettivo era una logica di combattimento deterministica e misurabile, completamente separata dalla grafica.

2.2.2.1 La macchina a turni

Problema. La battaglia deve risolvere i turni e, insieme, lasciare alla vista il controllo dei tempi dell'animazione. Mescolare le due cose legherebbe le regole ai millisecondi dello schermo.

Soluzione proposta. `Battaglia` è sincrona e istantanea: riceve un'`AzioneBattaglia`, risolve il turno intero (chi è più veloce attacca prima; a parità tocca al giocatore; si tira sulla precisione, si calcola il danno, si controlla l'esaurimento) e restituisce una lista immutabile di `EventoBattaglia` con i punti salute già aggiornati. Al ritorno lo stato è già quello finale: non conosce millisecondi, non attende, non produce frasi.

Alternative considerate. Far chiamare alla battaglia direttamente la vista (`vista.mostra(...)`) sarebbe stato più corto, ma avrebbe legato `gdr.battaglia` allo schermo, rendendo impossibile eseguire le battaglie senza finestra — e quindi anche la simulazione di bilanciamento. Scartata. Per l'avversario si è preferita una scelta della mossa a caso invece di un'IA: non fa sembrare il gioco truccato ed è l'unico punto da toccare se un giorno servisse della strategia.

Pattern applicati. Strategy / Dependency Injection: la sorgente casuale è un'interfaccia `Casuale` iniettata nel costruttore; `Battaglia` non chiama mai `Math.random()`, quindi il caso si può sostituire dall'esterno con un'implementazione deterministica (è così che sono state simulate le battaglie di bilanciamento). Comando: `AzioneBattaglia` è una `sealed interface` (`UsaMossa`, `Fuggi`) che incapsula la scelta del giocatore — la vista non calcola danni, costruisce solo il comando e lo passa.

2.2.2.2 Il calcolo del danno e l'efficacia elementale

Problema. Il danno deve dipendere da livello, potenza, attacco e difesa, con un po' di varietà, in modo ripetibile e testabile.

Soluzione proposta. `CalcolatoreDanno` è una utility senza stato (metodi `static`, costruttore privato): dati gli stessi Pokémon, la stessa mossa e la stessa sorgente casuale restituisce lo stesso `Risultato` (danno più efficacia). La formula è una variante di quella classica dei GDR a turni — cresce col livello e con la potenza, si divide per la difesa — con una variazione casuale fra l'85% e il 100% e un minimo di 1, così un colpo a segno fa sempre qualcosa. L'efficacia elementale vive in `Elemento`, in una tabella direzionale riempita in un blocco `static`.

Alternative considerate. Dare all'elettro una debolezza, per simmetria con gli altri tre elementi: scartata dopo averla misurata. L'unico Pokémon del giocatore è d'elettro e nessuno selvatico lo è, quindi una debolezza dell'elettro avrebbe punito solo il giocatore, e contro Verzino rendeva lo scontro di fatto imperdibile per l'avversario. Il triangolo è quindi FUOCO batte ERBA, ERBA batte ACQUA, ACQUA batte FUOCO, più ELETTRO forte su ACQUA: l'elettro senza debolezze è un'asimmetria voluta, il cui prezzo è che un secondo elemento per il giocatore richiederebbe di rivedere la tabella.

Pattern applicati. Utility *stateless* / funzione pura (`CalcolatoreDanno`); enum con comportamento (`Elemento.efficaciaContro`, che legge la tabella `[attaccante][difensore]`).

Il bilanciamento, misurato. I numeri del gioco (statistiche delle specie, formula del danno, moltiplicatori elementali) devono dare scontri vincibili ma non banali, e non si può decidere a occhio se lo siano. Poiché la battaglia è eseguibile senza interfaccia e con la sorgente casuale iniettata, è stata scritta — come strumento esterno al progetto — una simulazione che gioca molte migliaia di battaglie per ogni combinazione, pescando gli avversari dalle tabelle vere delle zone. I numeri non sono stimati, sono misurati. Con Scintillo (il Pokémon del giocatore) di livello 5 contro avversari di livello 5:

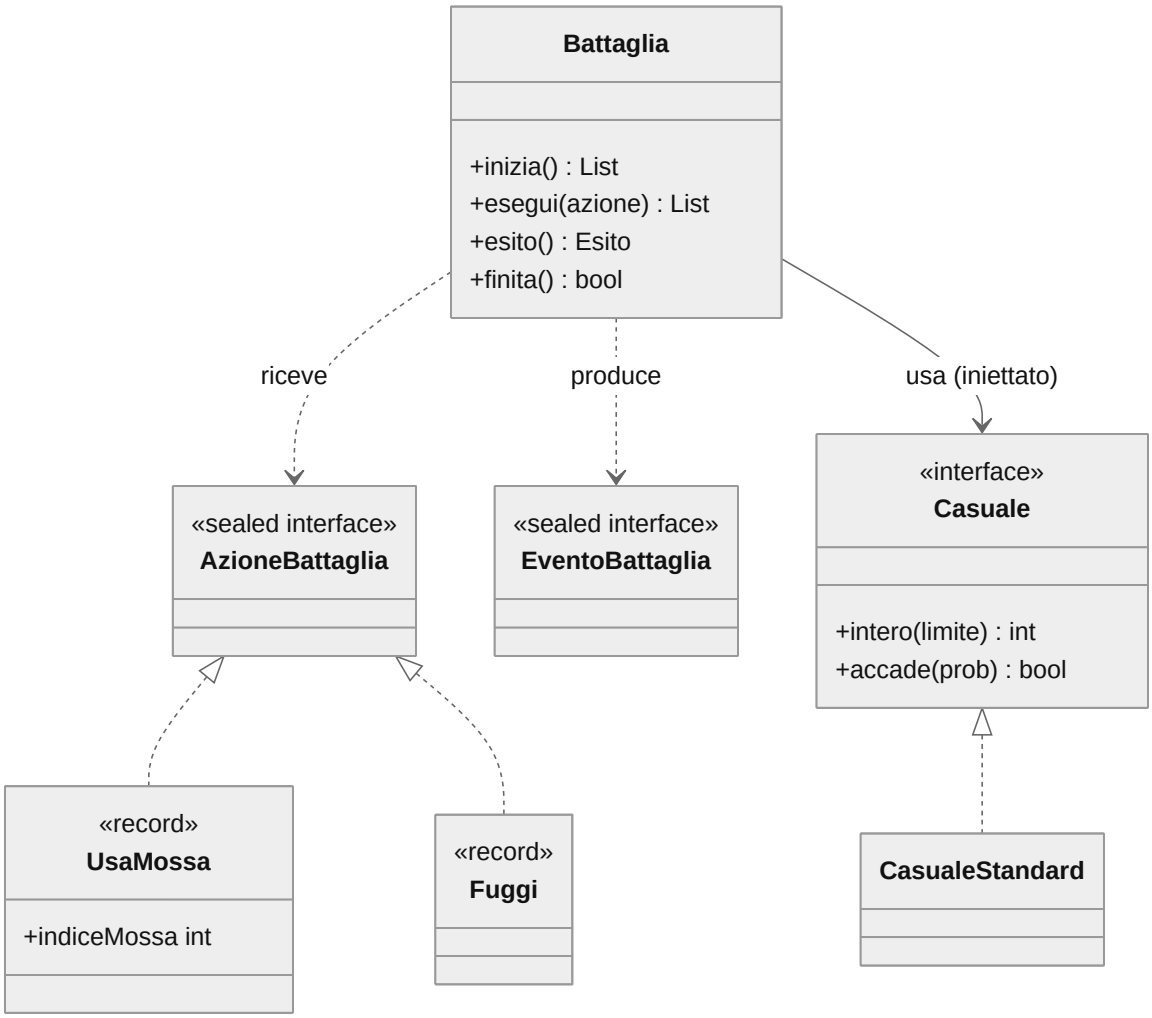


Figura 2.5: La battaglia: azione in ingresso, eventi in uscita, sorgente casuale iniettata.

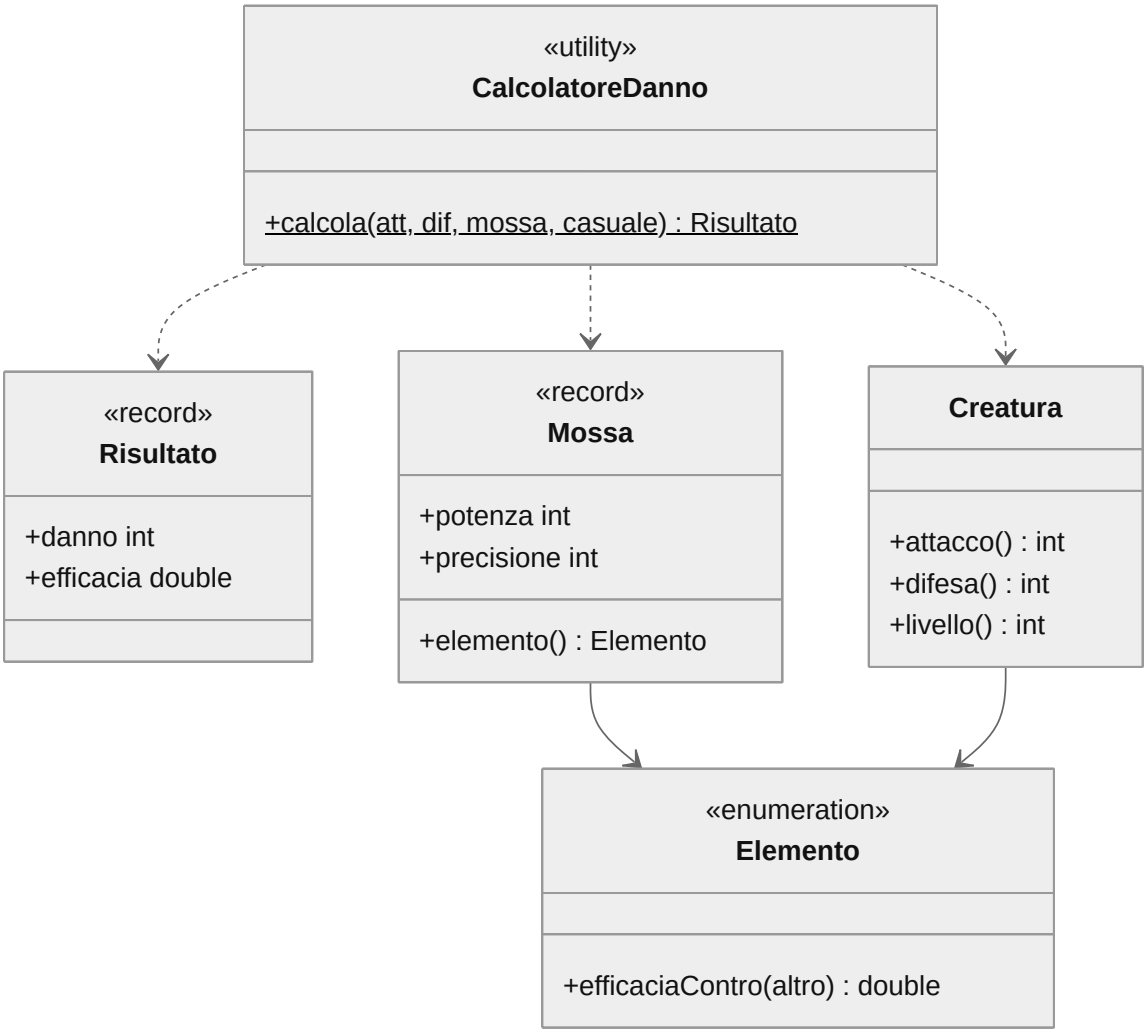


Figura 2.6: Il calcolo del danno e l'efficacia elementale.

Avversario	Vittorie	Turni medi
Rugiado	99,8%	5,0
Marezzo	99,6%	4,2
Fuligno	76,8%	6,2
Verzino	72,7%	8,1

E al variare del livello dell'avversario, dal 2 al 6 (il trattino indica i livelli che quella specie non raggiunge nelle tabelle delle zone):

Avversario	lv2	lv3	lv4	lv5	lv6
Rugiado	100%	100%	100%	100%	—
Marezzo	100%	100%	100%	100%	94%
Fuligno	100%	100%	100%	77%	25%
Verzino	—	—	100%	73%	51%

Due correzioni scoperte dalla misura. La prima: con una crescita di un decimo per livello, un Pokémon selvatico di livello 2 aveva quasi gli stessi punti salute del compagno di livello 5, e Verzino vinceva sempre; la crescita è passata a un quinto per livello. La seconda: Verzino (d'erba) restava imbattibile, perché l'erba resisteva all'elettro e lo colpiva al doppio mentre Scintillo attacca solo d'elettro; solo rimuovendo quella coppia di regole e riportando le statistiche di Verzino in linea con le altre lo scontro è salito al 75% di vittorie di media sui livelli con cui il bosco lo fa comparire (il 73% al livello 5 preso da solo). Ogni volta lo squilibrio l'ha scoperto la misura, non la lettura del codice.

Nota di metodo. È lo stesso principio dei test automatici: la casualità iniettata rende gli esperimenti ripetibili. La simulazione non fa parte del progetto consegnato, perché è uno strumento di sviluppo, non una funzione del gioco.

2.2.2.3 La generazione degli incontri con tregua

Problema. Gli incontri devono essere casuali ma non frustranti: non devono ripartire subito dopo una battaglia, e la frequenza non deve dipendere da quanti fotogrammi al secondo gira il gioco.

Soluzione proposta. `GeneratoreIncontri` è l'unico oggetto con memoria del package: conta i passi dall'ultimo incontro. La probabilità è per passo (12%), non al secondo, così non cambia con la velocità di camminata; dopo un incontro due passi sono sicuri (tregua), il terzo può già farne partire un altro. Il contatore sale a ogni passo concluso, anche fuori dall'erba alta: così uscendo dall'erba, facendo tre passi sul sentiero e rientrando, la tregua è già finita. Se scatta, pesca dalla `TabellaIncontri` della zona con un sorteggio in due tempi: prima la specie (stessa probabilità per tutte le voci della zona), poi il livello, uniforme nell'intervallo della `VoceIncontro`.

Alternative considerate. Una probabilità *al secondo* avrebbe reso gli incontri dipendenti dalla velocità e dagli FPS: scartata a favore della probabilità per passo. Pesi diversi per specie nella radura: provati e poi rimossi. Nella prima versione la radura aveva due sole specie, con Rugiado al 75%, e in dieci incontri se ne vedevano nove uguali: la zona sembrava avere un solo mostro. La soluzione non è stata ritoccare i pesi ma togliere il parametro e aggiungere una terza specie, tutte equiprobabili — un peso per voce sarebbe stato un numero da spiegare senza niente da esprimere. Verzino resta fuori dalla radura di proposito: è lo scontro più lungo del gioco e la radura è la zona in cui si impara a giocare.

Pattern applicati. Strategy (la stessa `Casuale` iniettata); record immutabili per i dati (`VoceIncontro`, `TabellaIncontri` con `List.copyOf` nel costruttore compatto).

2.2.3 Stefano Roberto Rusu — Grafica e orchestrazione

Mi sono occupato della grafica (`gdr.grafica`, 19 classi) e dell'orchestrazione dello stato (`gdr.partita.Partita`, `FaseGioco`, `EventoPartita`). Il compito era mettere in scena quello che la logica decide, senza mai farla dipendere dallo schermo.

2.2.3.1 Il ciclo a passo fisso e il confine aggiorna/disegna

Problema. Su monitor diversi il gioco deve andare alla stessa velocità, e la logica non deve mai disegnare né il disegno modificare lo stato.

Soluzione proposta. `CicloDiGioco` è un solo ciclo a passo fisso di 1/60 s: accumula il tempo realmente trascorso e fa avanzare la logica a passi costanti, con un tetto massimo perché una finestra rimasta ferma qualche secondo non recuperi centinaia di passi in una volta. `PannelloGioco` fa avanzare la logica in `aggiorna`, e in `paintComponent` disegna lo stato "com'è adesso" senza cambiarlo: chi aggiorna non disegna, chi disegna non aggiorna.

Alternative considerate. Un passo variabile legato al tempo reale renderebbe la fisica del passo diversa a ogni fotogramma: scartato a favore del passo fisso. Accumulare senza tetto: dopo una finestra ferma dieci secondi il ciclo girerebbe seicento volte di fila teletrasportando il personaggio; evitato con il tetto sull'accumulatore.

Pattern applicati. Game Loop con *fixed timestep*: un unico punto in cui il tempo entra nella simulazione, e una sola direzione di dipendenza fra aggiornamento e disegno.

2.2.3.2 La messa in scena della battaglia e le fasi di gioco

Problema. La logica risolve il turno tutto insieme (punti salute già finali), ma la barra dei PS deve scendere gradualmente e le frasi restare in scena il tempo giusto.

Soluzione proposta. `Partita` è la radice dello stato e fa da facciata alla logica per la vista; sa in che fase siamo fra `ESPLORAZIONE`, `DENTRO_OSPEDALE` e `BATTAGLIA`. `AnimatoreBattaglia` consuma gli eventi uno alla volta con le proprie durate e tiene dei valori *mostrati* distinti da quelli reali: la barra insegue a velocità costante un numero già fissato dalla logica. Finché c'è qualcosa in scena il giocatore non può scegliere. `Partita.chiudiBattaglia` viene chiamata dalla vista quando l'animazione è finita: la logica sa *cosa* succede, la vista sa *quando*.

Alternative considerate. Far scendere la barra dentro la logica, facendole restituire i punti salute un po' alla volta, avrebbe mescolato le regole con i tempi dello schermo e reso le battaglie non più risolvibili in un colpo — e quindi non più simulabili. Scartata. Scartata anche l'idea di far decidere alla logica quando chiudere la battaglia con un timer interno: avrebbe richiesto alla logica di conoscere i millisecondi.

Pattern applicati. Facade: `Partita` espone alla vista un'unica superficie coerente sopra tutti i package di logica. State: `FaseGioco` è un enum che determina quali input scorrono e quale schermata si disegna. Tutte le stringhe mostrate stanno in `Testi`, così tradurre il gioco vuol dire toccare un solo file.

2.2.3.3 Il renderizzatore e i fogli sprite

Problema. Ottenere l'aspetto "a pixel" anni '90 nitido, ridimensionabile senza deformare, da un insieme minimo di risorse, e dare varietà e movimento alla scena senza appesantire il codice.

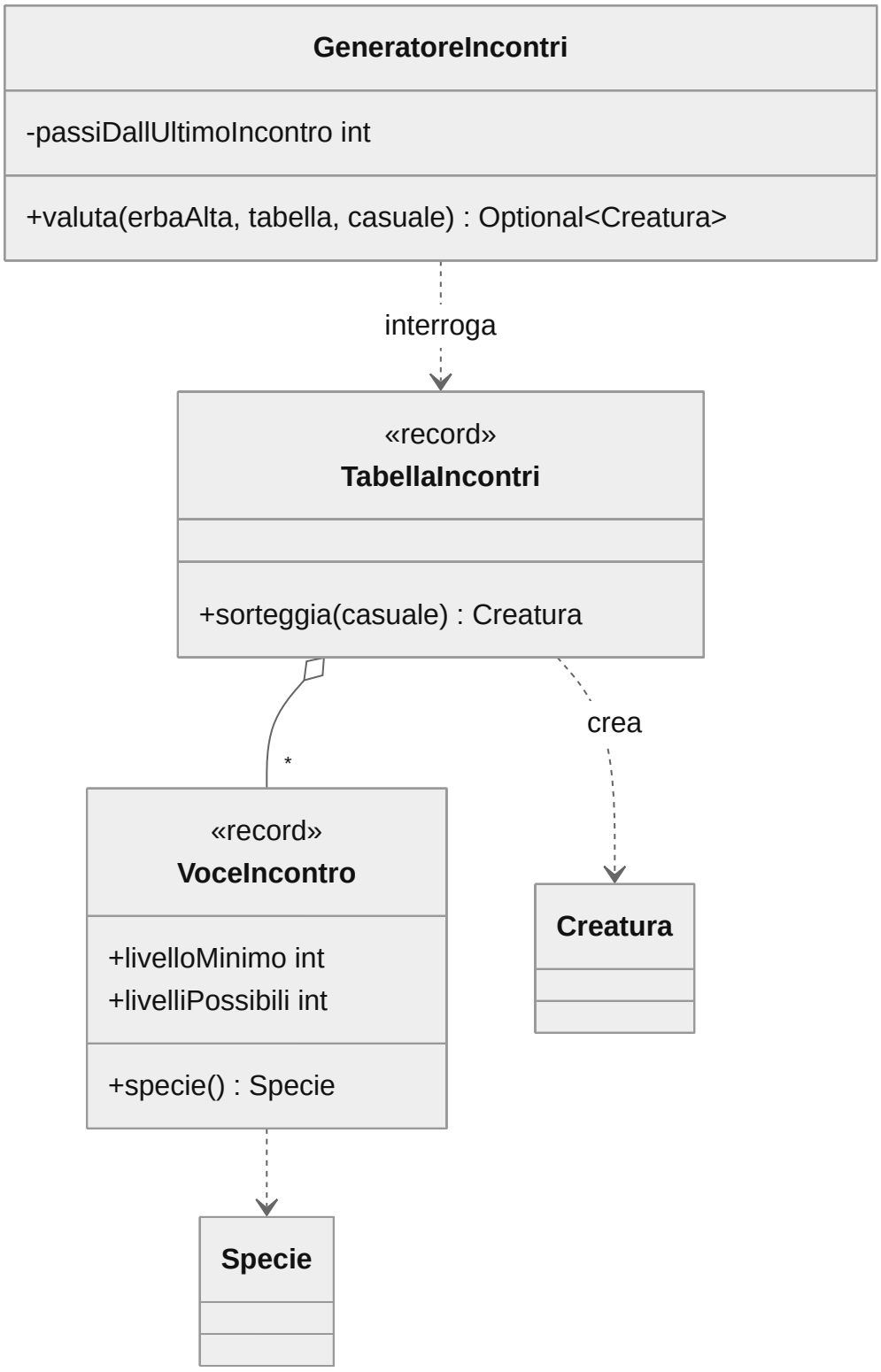


Figura 2.7: Gli incontri: conteggio dei passi, tregua e sorteggio in due tempi.

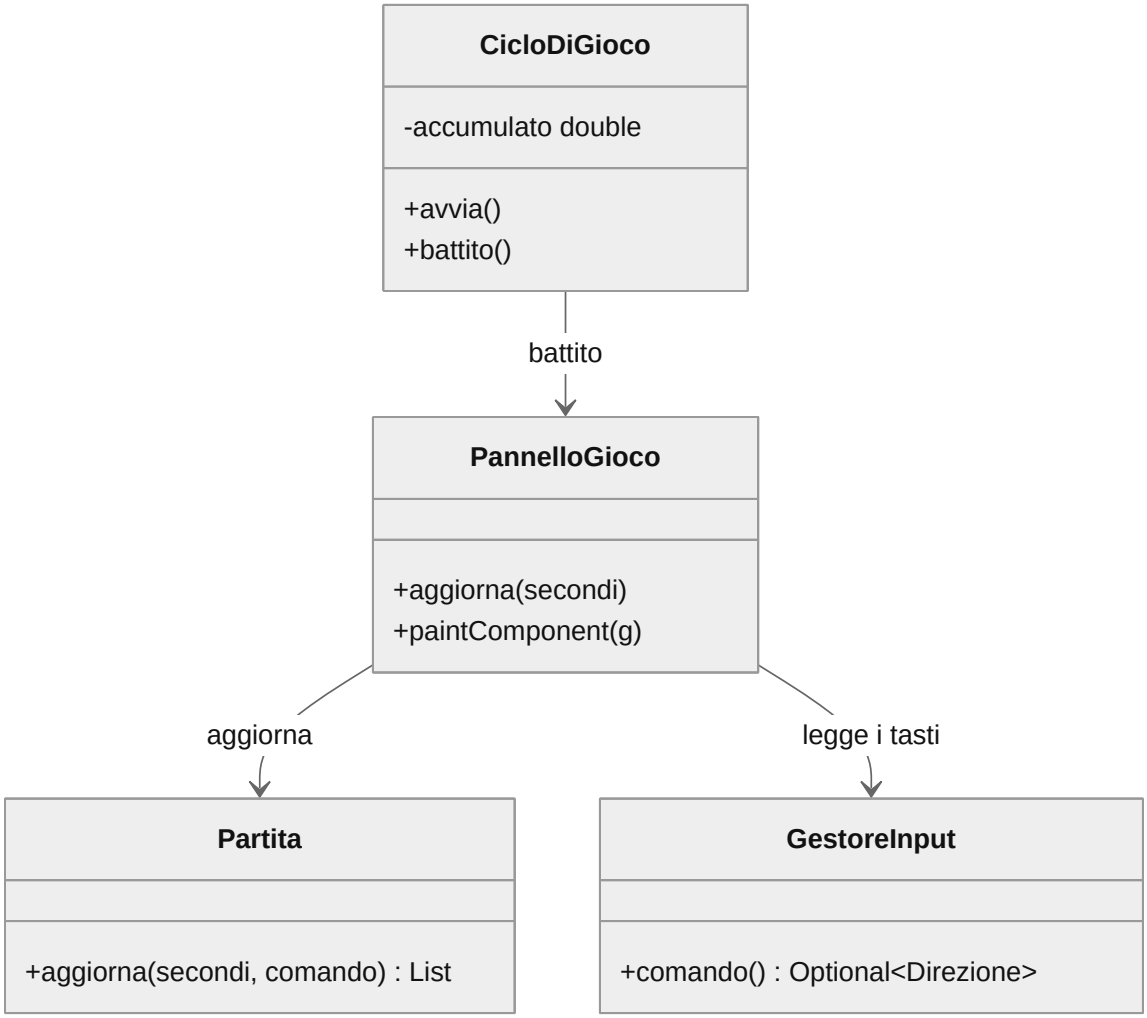


Figura 2.8: Il ciclo di gioco e il confine aggiorna/disegna.

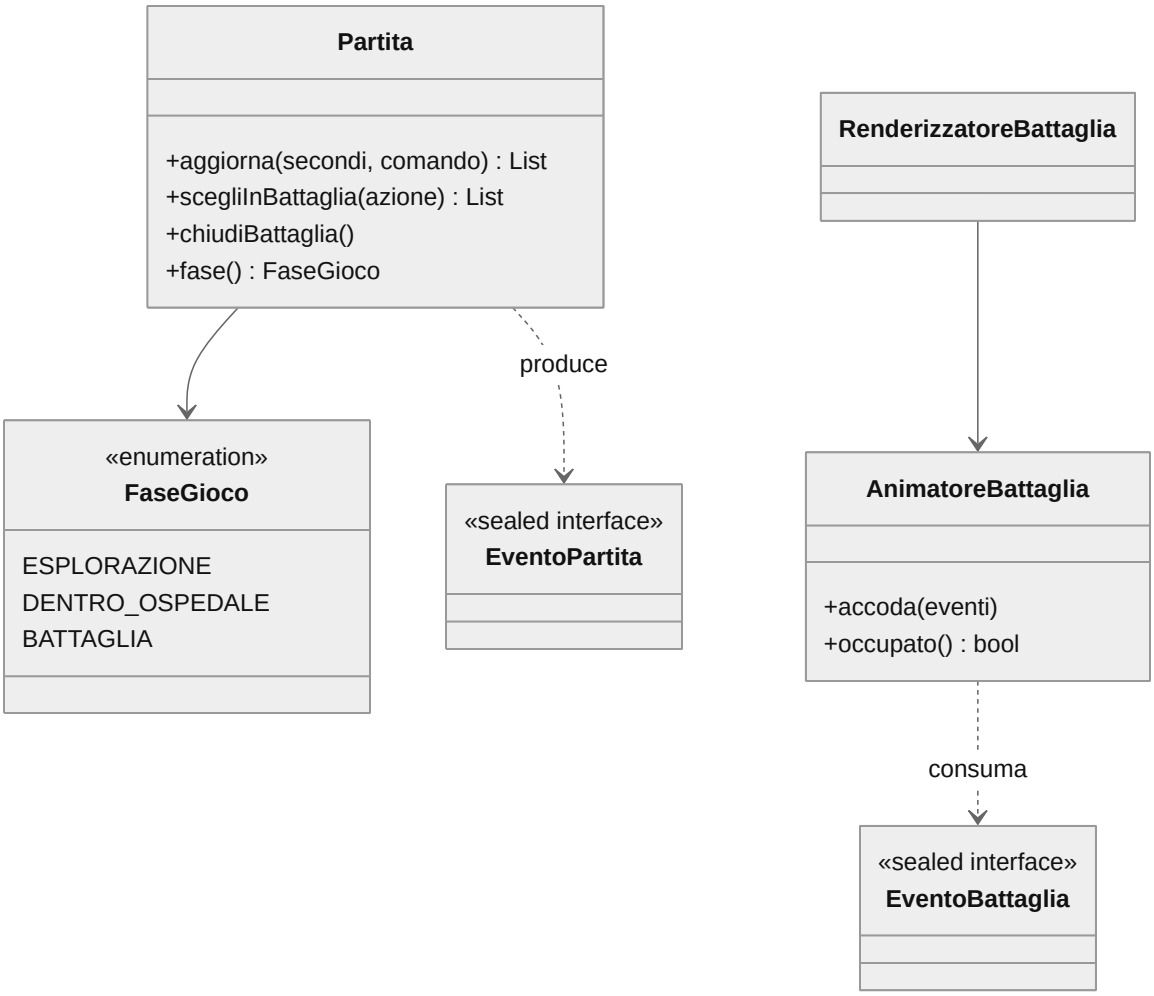


Figura 2.9: L'orchestrazione: fasi, eventi di partita e messa in scena.

Soluzione proposta. Si disegna sempre su una tela logica di 240x176 pixel (15x11 caselle da 16), poi ingrandita di un fattore intero. L'unico accorgimento esplicito è il suggerimento di rendering `VALUE_INTERPOLATION_NEAREST_NEIGHBOR`: senza di esso l'ingrandimento interpolerebbe i pixel e il risultato sarebbe sfocato invece che netto. `Telecamera` decide quale porzione di mappa mostrare, con un *clamp* ai bordi; i renderizzatori compongono la schermata; i disegnatori ritagliano i riquadri dai fogli PNG tramite `FoglioSprite`; `DisegnatoreEdificio` si appoggia a `ParteEdificio`, che ricava l'aspetto di una casella di edificio contando i vicini, così non esiste una seconda struttura dati da tenere allineata alla mappa.

Tutta la grafica viene da tre fogli PNG (`terreno`, `personaggio`, `creature`), letti e tenuti in cache da `FoglioSprite`: sostituire un disegno vuol dire sostituire un riquadro nel PNG senza toccare il codice, e aggiungere una specie è aggiungere un riquadro in coda più il suo nome in `DisegnatoreCreatura.NOMI`. L'acqua, l'unica cosa animata, non ha il movimento nel PNG: sono sedici fotogrammi e il codice sceglie quale mostrare con `Math.floorMod((long)(tempo*7)+riga*3, 16)`, sfasando le righe di proposito perché il laghetto non pulsi tutto insieme.

Alternative considerate. Disegnare terreno, edifici e personaggi a mano con `Graphics2D`, come nella prima versione, evitava i file ma costava molto più codice per un risultato peggiore: la conversione ai fogli sprite ha tolto righe e ha reso modificabile l'aspetto senza ricompilare. Scartato anche l'ingrandimento a fattore frazionario: i pixel dei tile risulterebbero di dimensioni diverse fra loro.

Pattern applicati. Nessuno, ed è una risposta e non un'omissione: qui non c'è un problema di struttura da risolvere con un pattern noto, ma un problema di dati. La cache dei fogli in `FoglioSprite` è un'ottimizzazione (un foglio letto una volta sola e condiviso), non un pattern di progettazione, e `ParteEdificio` è una funzione pura dalla mappa all'aspetto.

Nota sulla testabilità. Cinque classi di `gdr.grafica` non importano AWT — `Telecamera`, `VeloTransizione`, `ParteEdificio`, `Testi` e `AnimatoreBattaglia` — e sono quindi collaudabili senza aprire una finestra: stanno in quel package perché servono solo alla vista, non perché disegnino.

Nota. L'ordine delle costanti conta in due soli punti del progetto — la riga di `personaggio.png` segue `sguardo.ordinal()`, la colonna di `creature.png` segue l'ordine dei nomi — gli unici due punti in cui una posizione nel foglio deve restare allineata al codice.

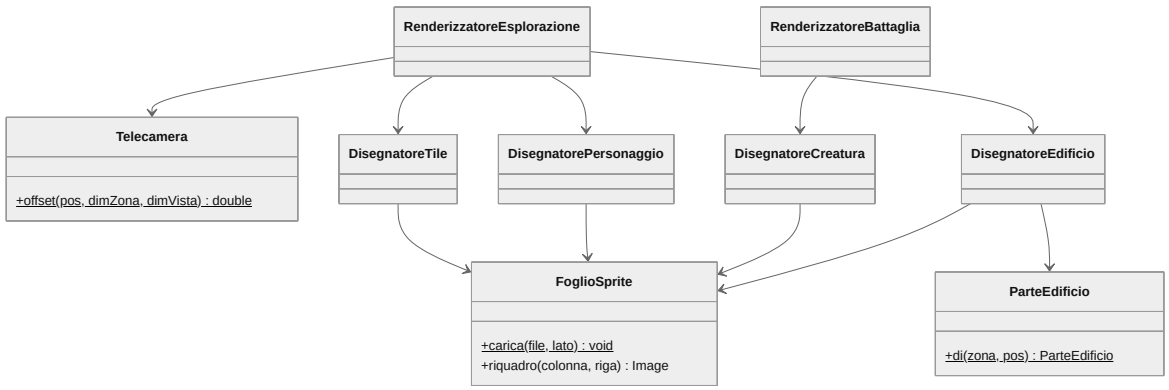


Figura 2.10: Il rendering: telecamera, renderizzatori, disegnatori e fogli sprite.

2.2.3.4 La porta dell'ospedale senza interni

Problema. Gli interni degli edifici sono fuori scope e nessuna porta è calpestabile; eppure l'ospedale deve curare. Serviva una cura senza costruire interni.

Soluzione proposta. Sbattere contro una porta produce un evento (`PortaToccata`) invece di non fare nulla; `Partita` lo intercetta e, se la porta è quella dell'ospedale, entra nella fase `DENTRO_OSPEDALE` per qualche secondo: il personaggio non viene disegnato, l'input è ignorato e un riquadro dice cosa succede; a fine visita il Pokémon è curato e si torna a `ESPLORAZIONE`. L'evento esce al primo contatto e non più finché si resta premuti contro la porta, altrimenti uscirebbe sessanta volte al secondo e si verrebbe curati in ciclo. Durante la visita `Partita` passa comunque il comando all'esplorazione con `osservaSenzaMuovere`, così un tasto lasciato dentro non impedisce di rientrare.

Alternative considerate. Costruire una vera mappa interna per l'ospedale sarebbe stato il modo "giusto", ma avrebbe richiesto una seconda categoria di zone, il salvataggio del punto di rientro e dei tile d'interno: fuori scope per il traguardo. Scartata anche l'idea di curare sul posto senza fase dedicata: senza i tre secondi e il riquadro, la cura sarebbe stata invisibile al giocatore.

Perché conviene. Aggiungere una funzione è stato aggiungere un evento e un caso di fase, non una nuova struttura dati: la mappa resta testo, e aggiungere un secondo ospedale è scrivere un'altra `H`.

Pattern applicati. State (`FaseGioco`) e comunicazione a eventi.

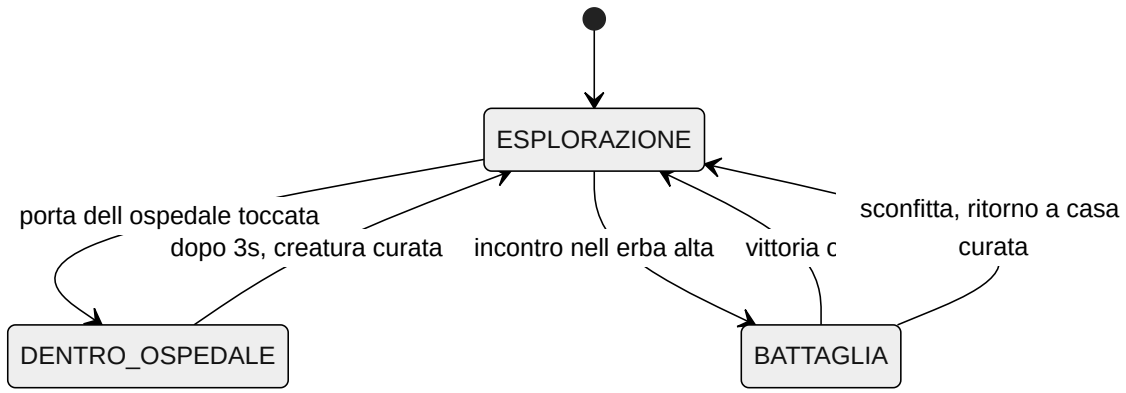


Figura 2.11: Le tre fasi di gioco e le transizioni fra loro.

2.2.3.5 Le trappole dell'input in Swing

Problema. In Swing l'input da tastiera passa per le `InputMap`/`ActionMap`, con comportamenti non ovvi; tre difetti compilavano senza un solo avviso e si vedevano solo giocando.

Soluzione proposta. `GestoreInput` risolve tre trappole. (1) Una `InputMap` associa una combinazione a una sola azione: le frecce su e giù, usate sia per camminare sia per scorrere il menu di battaglia, sono gestite da un'unica azione che fa entrambi i lavori, invece di due registrazioni che si annullavano. (2) L'elenco dei tasti premuti contiene i nomi dei tasti e non le direzioni, perché ogni direzione ha due tasti (freccia e lettera WASD) e la direzione si ricava dall'ultimo tasto ancora premuto. (3) Ogni rilascio è registrato anche nelle varianti con modificatore (shift, ctrl, alt), perché una `InputMap` confronta la combinazione esatta e un `released` non scatterebbe se nel frattempo è premuto un modificatore, lasciando il personaggio a camminare da solo.

Alternative considerate. Un `KeyListener` diretto darebbe pieno controllo sugli eventi grezzi, ma perderebbe la risoluzione dei conflitti e il legame con il fuoco che le `InputMap` offrono, e andrebbe riscritto a mano: scartato. Resta un limite noto, dichiarato nel codice: un rilascio avvenuto mentre la finestra non ha il fuoco non arriva comunque, e per accorgersene servirebbe ascoltare il fuoco della finestra, che da `GestoreInput` non si raggiunge.

Nota. Non è un pattern ma una lezione di libreria, documentata nel codice: in Swing, se un tasto deve servire a due scopi va registrato una volta sola.

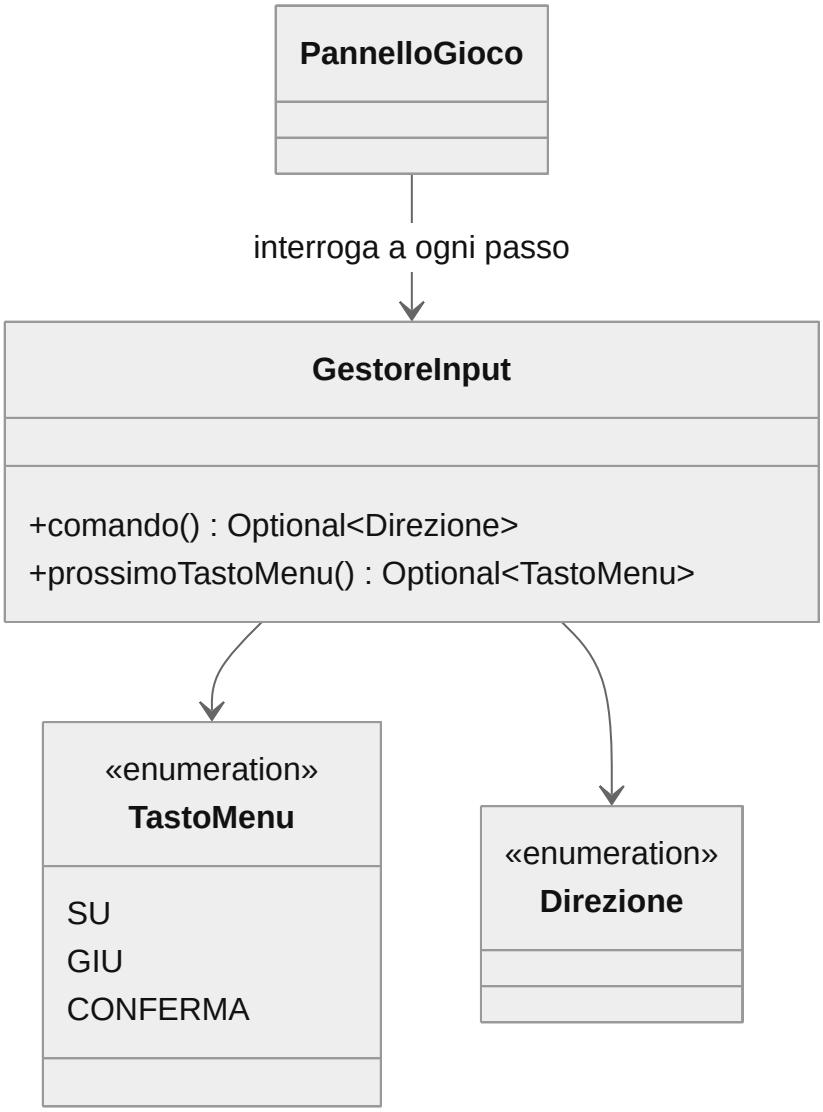


Figura 2.12: L'input: tasti premuti per nome, direzione ricavata, coda separata per il menu.

Capitolo 3

Sviluppo

3.1 Testing automatizzato

Il collaudo automatico copre tutta la logica di gioco: regole della battaglia, lettura del mondo, movimento a griglia, generazione degli incontri e orchestrazione delle fasi. Non copre il disegno, che per sua natura va guardato.

Questa divisione non è casuale: la logica è stata progettata dall'inizio per essere collaudabile senza interfaccia grafica, cioè in classi che non importano né AWT né Swing e che non sanno di esistere dentro una finestra; su quella base è stata poi scritta una suite JUnit 5. Delle 50 classi del progetto, 36 non importano AWT — le 31 dei package di logica più cinque di `gdr.grafica` — e sono quindi collaudabili da riga di comando; le 14 restanti disegnano o gestiscono la finestra, e per quelle il collaudo è stato manuale.

3.1.1 La suite

13 file, 2231 righe, 112 prove, tutte verdi (JUnit 5, Eclipse):

Classe di prova	Prove	Cosa verifica
`PosizioneTest`	5	coordinate e direzioni: l'asse delle righe cresce verso il basso, andata e ritorno, immutabilità
`MovimentoGrigliaTest`	13	un passo dura 15 aggiornamenti, la casella resta intera durante il transito, un solo passo per aggiornamento
`LettoreZonaTest`	10	ogni simbolo della legenda, calpestabilità, mappa storta o simbolo ignoto rifiutati con messaggio
`MondoTest`	10	passaggi fra zone, coerenza dei collegamenti, punto di rientro
`ElementoTest`	6	tabella delle efficacie, simmetria e casi neutri
`CreaturaTest`	10	crescita per livello, PS che non scendono sotto zero, cura
`CalcolatoreDannoTest`	7	formula del danno, almeno un punto per colpo a segno, efficacia
`BattagliaTest`	13	ordine dei turni, pareggio di velocità, fuga, vittoria, sconfitta
`GeneratoreIncontriTest`	10	probabilità nell'erba alta, i due passi sicuri dopo un incontro, scelta della specie
`EsplorazioneTest`	12	collisioni, cambio zona, porta dell'ospedale annunciata una volta sola

Classe di prova	Prove	Cosa verifica
`PartitaTest`	16	transizioni fra le tre fasi, cura in ospedale, rifiuto dei tempi non validi
Totale	**112**	

A queste si aggiungono due classi di supporto che non contengono prove: `CasualeDiProva`, un generatore casuale pilotato che restituisce i numeri decisi dalla prova (nessuna classe di logica chiama `Math.random()`, ma riceve un `Casuale`), e `MappeDiProva`, che costruisce un mondo sintetico di due zone con Pokémon dalle statistiche scelte a tavolino (una veloce, una lenta, una pari). Le prove non usano le mappe vere del gioco: così un ritocco alla mappa non rompe il collaudo, e viceversa.

3.1.2 Come verificare che le prove servano davvero

Una suite tutta verde non dimostra niente da sola: dimostra qualcosa solo se fallisce quando il codice è sbagliato. Per verificarlo abbiamo introdotto otto difetti deliberati, uno alla volta, nei punti più delicati, rieseguendo tutta la suite dopo ognuno. Ogni difetto è stato scoperto da almeno una prova:

Difetto introdotto	Prove che lo scoprono
La porta dell'ospedale si riannuncia a ogni aggiornamento	5
La crescita per livello è un decimo invece di un quinto	2
A parità di velocità il turno va all'avversario	1
Un colpo a segno può fare zero danni	1
I punti salute possono scendere sotto zero	1
Nessuna tolleranza nel confronto del progresso con 1	12
La tregua dopo un incontro sparisce	1
Il passaggio fra zone viene annunciato anche come passo normale	3

Il difetto sulla tolleranza è quello che fa cadere più prove (dodici), ed è coerente con quanto detto nel capitolo precedente: è un errore di un milionesimo che, moltiplicato per ogni passo, rallenterebbe il gioco di un sedicesimo senza che nessun messaggio d'errore lo segnali.

3.1.3 Come si eseguono

In Eclipse: tasto destro sulla cartella `test` > Run As > JUnit Test. La cartella `test` è dichiarata come sorgente di prova nel `.classpath` del progetto, insieme alla libreria JUnit 5, quindi l'importazione del progetto non richiede configurazioni aggiuntive.

3.2 Metodologia di lavoro

Il lavoro è stato diviso per package, non per fasi: ognuno ha avuto la responsabilità di una parte verticale del progetto, dalla progettazione alle prove. La divisione è leggibile direttamente nella struttura del codice, e questo non è un caso: era il criterio con cui l'abbiamo scelta.

Il confine fra le parti è sempre un'interfaccia o un tipo di dato immutabile, mai una chiamata diretta a un dettaglio interno:

- fra la logica del mondo e la grafica c'è `Zona`, che espone solo domande (`calpestabile`, `generaIncontri`, `tipo`);
- fra la logica e la vista ci sono le tre famiglie di eventi (`EventoPartita`, `EventoEsplorazione`, `EventoBattaglia`, tutte `sealed interface`), liste immutabili prodotte da un lato e consumate dall'altro; nel verso opposto l'unica cosa che entra è `AzioneBattaglia`, sigillata allo stesso modo;
- fra chiunque abbia bisogno del caso e il caso c'è l'interfaccia `Casuale`.

Grazie a questi confini le tre parti si sono potute sviluppare e collaudare in parallelo: la battaglia era simulabile con `CasualeDiProva` prima che esistesse una schermata di battaglia, e la grafica era avviabile su un mondo di prova prima che le mappe definitive fossero scritte.

Il lavoro di integrazione. L'integrazione delle parti è stata fatta a mano: ci siamo scambiati le cartelle e abbiamo ricomposto il progetto, ricompilando ed eseguendo l'intera suite di prove dopo ogni unione. Il costo è stato basso per una ragione precisa: i confini elencati sopra sono tipi, non chiamate, quindi un'unione sbagliata non compilava invece di produrre un gioco che si comportava male. Gli unici file toccati da più di una persona sono stati quelli sul confine — le interfacce sigillate degli eventi, `Casuale`, `Zona` — e ogni modifica a quei file è stata concordata prima di scriverla.

3.2.1 Tonino Zulli

Responsabile di `gdr.nucleo`, `gdr.entita`, `gdr.mondo`, della logica di esplorazione (`gdr.partita.Esplorazione` con i suoi eventi) e delle mappe in `gdr.dati.ZoneIniziali`: coordinate e direzioni, movimento a griglia, lettura e validazione delle mappe testuali, collisioni e collegamenti fra zone.

Prove scritte: `PosizioneTest`, `MovimentoGrigliaTest`, `LettoreZonaTest`, `MondoTest` (38 prove), più il mondo sintetico `MappeDiProva` usato da tutti.

In autonomia: `Posizione`, `Direzione`, `MovimentoGriglia`, `Giocatore`, `LettoreZona`, `Zona`, `Mondo`, `Passaggio`, `TipoTile`, `Esplorazione` e `EventoEsplorazione`, le mappe di `ZoneIniziali`, e le quattro classi di prova elencate sopra.

In collaborazione: la forma di `EventoEsplorazione` e i suoi casi, concordata con Stefano Roberto Rusu che li consuma in `Partita`; l'interfaccia `Casuale`, concordata con Agyiri Godfred Yeboah che la usa in battaglia; il mondo sintetico `MappeDiProva`, scritto da me ma su richieste di tutti e tre, perché serve anche alle prove di battaglia e di partita.

3.2.2 Agyiri Godfred Yeboah

Responsabile di `gdr.battaglia`, `gdr.incontri` e dei contenuti in `gdr.dati.Bestiario` e `gdr.dati.Repertorio`: statistiche dei Pokémon, tabella delle efficacie, calcolo del danno, ordine dei turni, fuga, generazione degli incontri e tregua.

Prove scritte: `ElementoTest`, `CreaturaTest`, `CalcolatoreDannoTest`, `BattagliaTest`, `GeneratoreIncontriTest` (46 prove), più il generatore pilotato `CasualeDiProva` e le simulazioni di bilanciamento su centinaia di migliaia di battaglie.

In autonomia: `Creatura`, `Specie`, `Mossa`, `Elemento`, `Esito`, `CalcolatoreDanno`, `Battaglia`, `AzioneBattaglia`, `EventoBattaglia`, `GeneratoreIncontri`, `TabellaIncontri`, `VoceIncontro`, i contenuti di `Bestiario` e `Repertorio`, le cinque classi di prova elencate sopra e il generatore pilotato `CasualeDiProva`.

In collaborazione: l'interfaccia `Casuale` (con Tonino Zulli, che la ospita in `gdr.nucleo`); l'elenco e i campi di `EventoBattaglia`, concordati con Stefano Roberto Rusu, che da quegli eventi

ricava le frasi e le durate in scena — ogni evento aggiunto o modificato è passato per un accordo, perché rompe la compilazione della vista; le tabelle incontri delle due zone, decise insieme a Tonino Zulli che possiede le mappe.

3.2.3 Stefano Roberto Rusu

Responsabile di `gdr.grafica` e dell'orchestrazione dello stato (`gdr.partita.Partita`, `FaseGioco`, `EventoPartita`): ciclo a passo fisso, finestra e input, telecamera, renderizzatori e disegnatori, fogli sprite, messa in scena della battaglia e fasi di gioco.

Prove scritte: `EsplorazioneTest`, `PartitaTest` (28 prove). La parte di disegno è stata collaudata manualmente, perché il suo risultato è un'immagine.

In autonomia: le 19 classi di `gdr.grafica` (ciclo, finestra, pannello, input, telecamera, renderizzatori, disegnatori, fogli sprite, animatore, testi, velo di transizione, parte edificio), `Partita`, `FaseGioco`, `EventoPartita` e le due classi di prova elencate sopra.

In collaborazione: `EventoBattaglia` con Agyiri Godfred Yeboah e `EventoEsplorazione` con Tonino Zulli, per le ragioni già dette; la fase `DENTRO_OSPEDALE`, che ha richiesto un evento nuovo (`PortaToccata`) nella parte di Tonino Zulli e un accordo su quando l'evento debba uscire — al primo contatto e non a ogni aggiornamento.

3.2.4 Strumenti

- Eclipse come ambiente di sviluppo; il progetto è importabile così com'è (`.project` e `.classpath` fanno parte della consegna, con la cartella `test` già dichiarata come sorgente di prova).
- JUnit 5 per le prove automatiche.

3.3 Note di sviluppo

Elenco delle feature avanzate del linguaggio e dell'ecosistema Java effettivamente usate, con il punto preciso in cui si trovano. Dove una categoria non è rappresentata lo diciamo invece di tacerlo.

3.3.1 Tonino Zulli

- **Generici.** Nessun tipo generico scritto da noi in questi package: le strutture dati sono tutte specifiche (una griglia `TipoTile[][]`, una `List<Passaggio>`), e l'uso di classi generiche di libreria non conta come avanzato.
- **Lambda expressions.** Nessuna in `gdr.nucleo`, `gdr.mondo`, `gdr.entita`.
- **Optional e costrutti funzionali.**
  - `Zona.passaggioIn(Posizione)` restituisce `Optional<Passaggio>`: una casella può avere o non avere un passaggio, e il tipo lo dichiara invece di affidarsi a un null da ricordare.
  - `Esplorazione.aggiorna(secondi, comando)` riceve un `Optional<Direzione>`: "nessun tasto premuto" è un valore, non un riferimento nullo.
- **Algoritmi non forniti da alcuna libreria.**
  - *Movimento a griglia interpolato.* La posizione logica resta una casella intera; un progresso fra 0 e 1 serve solo al disegno. Due accorgimenti non banali: un aggiornamento conclude al massimo un passo (l'eccedenza di tempo si scarta, altrimenti un singolo aggiornamento potrebbe attraversare due caselle e quindi due zone), e il confronto del progresso con 1 usa una tolleranza di `1e-9`, senza la quale

la somma di quindici quindicesimi in virgola mobile non raggiunge 1 e ogni passo dura 16 aggiornamenti invece di 15.
  - *Lettura e validazione delle mappe testuali.* Da liste di stringhe a griglia rettangolare, con l'errore riportato per posizione: nome della mappa, riga, colonna e simbolo incriminato.
  - *Verifica degli invarianti del grafo delle zone,* una sola volta all'avvio: ogni passaggio punta a una zona esistente; la casella d'arrivo è dentro la zona di destinazione ed è calpestabile; l'arrivo non è a sua volta un passaggio, altrimenti si rimbalzerebbe all'infinito; la zona di destinazione ha un passaggio che riporta indietro.

3.3.2 Agyiri Godfred Yeboah

- **Generici: un generico *bounded* scritto da noi.** `BattagliaTest` definisce `private static <T extends EventoBattaglia> T come(Class<T> tipo, EventoBattaglia evento)`, che verifica il tipo di un evento e lo restituisce già convertito, togliendo un cast da ognuna delle decine di asserzioni che lo usano. È l'unico tipo generico nostro del progetto.
- **Lambda expressions.** Nessuna in questi package: la sorgente casuale è un'interfaccia con due metodi astratti, quindi non è un'interfaccia funzionale e viene implementata da classi.
- **Optional e costrutti funzionali.**
  - `GeneratoreIncontri.valuta(erbaAlta, tabella, casuale)` restituisce `Optional<Creatura>`: "nessun incontro" è l'`Optional` vuoto, non un null da controllare.
- **Algoritmi non forniti da alcuna libreria.**
  - *Formula del danno.* Variante della formula classica dei GDR a turni: cresce con il livello dell'attaccante e con la potenza della mossa, si divide per la difesa del bersaglio, si moltiplica per l'efficacia elementale, poi una variazione casuale fra l'85% e il 100% e un minimo di 1, così un colpo a segno fa sempre qualcosa.
  - *Tabella direzionale delle efficacie* `[attaccante][difensore]`, riempita in un blocco `static` con due sole operazioni (forte, debole): la tabella è il dato, non una catena di condizioni.
  - *Risoluzione del turno in una chiamata sincrona:* ordine per velocità con il pareggio assegnato al giocatore, tiro sulla precisione, calcolo del danno, controllo dell'esaurimento. Al ritorno lo stato è già quello finale, il che rende le battaglie simulabili a migliaia.
  - *Sorteggio dell'incontro in due tempi* — prima la specie, uniforme fra le voci della zona, poi il livello, uniforme nell'intervallo di quella voce — governato da un contatore di passi che impone due passi sicuri dopo ogni incontro e che avanza anche fuori dall'erba alta.

3.3.3 Stefano Roberto Rusu

- **Generici.** Nessun tipo generico scritto da noi in `gdr.grafica`: le strutture usate sono quelle di libreria (`List`, `Optional`), che non contano come avanzate.
- **Lambda expressions.** Tutte in `gdr.grafica`:
  - `CicloDiGioco`: `new Timer(RITARDO_TIMER, evento -> battito())`, l'`ActionListener` del timer di Swing.
  - `GestoreInput`: l'effetto di ogni tasto è passato come `Runnable` (`() -> premuti.remove(tasto)`, `() -> accoda(tastoMenu)`), così la registrazione del tasto e ciò che fa restano separate.

  - `FoglioSprite`: `CARICATI.computeIfAbsent(nome, chiave -> leggi(chiave, lato))`, che legge il PNG solo la prima volta che serve.
- **Optional e costrutti funzionali.**
  - `GestoreInput.comando()` è `Optional.ofNullable(premuti.peekLast())` seguito da `.map(direzioni::get)`: `Optional` più *method reference*; nessun tasto premuto dà un `Optional` vuoto, e il nome del tasto viene tradotto in direzione solo se un tasto c'è davvero. Il `map` non fa dimenticare il controllo sul nullo.
  - `SwingUtilities.invokeLater(Avvio::costruisciEAvvia)`: *method reference* per costruire l'interfaccia sull'Event Dispatch Thread.
  - `GestoreInput.prossimoTastoMenu()` restituisce `Optional<TastoMenu>`.
- **Algoritmi non forniti da alcuna libreria.**
  - *Ciclo a passo fisso.* Si accumula il tempo realmente trascorso e si fa avanzare la logica a passi costanti di 1/60 s, con un tetto sull'accumulatore: senza quel tetto, una finestra rimasta ferma dieci secondi farebbe girare il ciclo seicento volte di fila teletrasportando il personaggio.
  - *Composizione dell'aspetto degli edifici.* `ParteEdificio.di(zona, posizione)` ricava quale pezzo disegnare contando i vicini della casella, quindi non esiste una seconda struttura dati da tenere allineata alla mappa: cambiare la mappa cambia da solo il disegno.
  - *Telecamera con clamp.* La porzione di mappa visibile insegue il personaggio ma non esce mai dai bordi della zona.
  - *Animazione dell'acqua.* `Math.floorMod((long)(tempo * 7) + riga * 3, 16)`: il movimento non sta nel PNG ma nella scelta del fotogramma, e lo sfasamento per riga impedisce al laghetto di pulsare tutto insieme.
  - *Valori mostrati contro valori reali.* `AnimatoreBattaglia` fa inseguire alla barra dei punti salute, a velocità costante, un numero che la logica ha già fissato: è ciò che permette di animare senza che la logica conosca i millisecondi.

3.3.4 Nota comune

- Nel codice di produzione non ci sono Stream né tipi generici scritti da noi (l'unico generico, un helper *bounded*, sta nelle prove): una scelta, non una dimenticanza. Le strutture dati sono tutte specifiche, le poche ricerche che facciamo sono più chiare come cicli, e in un ciclo a 60 aggiornamenti al secondo abbiamo preferito non creare oggetti intermedi a ogni fotogramma.
- I pattern di progettazione non sono elencati qui ma nella sezione *Design dettagliato*, dove ognuno è descritto insieme al problema che risolve.

Capitolo 4

Commenti finali

4.1 Autovalutazione e lavori futuri

Il traguardo che ci eravamo dati è raggiunto: due zone esterne collegate da un passaggio, movimento a griglia con animazione fluida, erba alta che genera incontri casuali e battaglia a turni funzionante, più una cosa che non era in programma, l'ospedale che cura. Il risultato sono 50 classi e 4155 righe, con 2231 righe di prove automatiche.

Cosa riteniamo riuscito.

Il confine fra logica e grafica ha tenuto fino alla fine, e si vede da un fatto misurabile: 36 classi su 50 non importano AWT, e la battaglia è simulabile centomila volte di fila senza aprire una finestra. È ciò che ci ha permesso di misurare il bilanciamento invece di indovinarlo, e di collaudare le regole mentre la grafica era ancora provvisoria.

Le gerarchie `sealed` sono state l'altra scelta che ha pagato: ogni volta che abbiamo aggiunto un evento, il compilatore ci ha detto esattamente dove andava gestito. Diverse funzioni (il passaggio fra zone, l'ospedale, il velo di transizione) sono state aggiunte senza modificare le strutture dati, solo aggiungendo un evento e i casi corrispondenti.

Cosa non siamo riusciti a fare.

- Gli interni degli edifici sono fuori scope, come dichiarato dall'inizio: le porte non sono calpestabili e l'ospedale cura con una fase dedicata invece che con una mappa interna. Aggiungerli richiederebbe una seconda categoria di zone e il salvataggio del punto di rientro.
- Il collaudo automatico del disegno non esiste: le 14 classi che importano AWT sono state verificate guardando il gioco. Un confronto automatico fra immagini di riferimento e fotogrammi prodotti sarebbe la strada naturale, ma richiederebbe di aggiungere alla consegna immagini di riferimento come asset binari.
- Il salvataggio della partita non c'è: ogni avvio ricomincia dall'inizio. Lo stato è però tutto raccolto in `Partita` e composto da oggetti immutabili o quasi, quindi serializzarlo sarebbe un lavoro contenuto.
- Nessuna progressione e un solo compagno: vincere non dà niente. Ogni specie conosce due mosse (`Specie.MOSSE_PER_SPECIE`) e il menu di battaglia le mostra entrambe più la fuga, ma il Pokémon del giocatore resta al livello 5 per tutta la partita: la formula di crescita per livello c'è ed è usata per generare i Pokémon selvatici ai livelli delle tabelle di zona, mentre il compagno non guadagna esperienza, non impara mosse nuove e non si può sostituire. Era la funzione successiva in programma e non c'è stato tempo.

Un limite noto e dichiarato. Un rilascio di tasto avvenuto mentre la finestra non ha il fuoco non arriva a `GestoreInput`, che quindi continua a considerare quel tasto premuto. È documentato nel codice: per risolverlo servirebbe ascoltare il fuoco della finestra, che da quella classe non si raggiunge.

4.2 Difficoltà incontrate

Le difficoltà vere non sono state algoritmiche ma di confine: decidere chi sa cosa.

La più istruttiva è stata la barra dei punti salute. La logica risolve il turno tutto in una volta, perché solo così le battaglie sono simulabili e collaudabili; ma sullo schermo la barra deve scendere gradualmente. La prima versione aveva provato a far scendere i punti salute dentro la logica, e il risultato era che la logica aveva cominciato a conoscere i millisecondi — e le prove avevano cominciato a dover aspettare. La soluzione (valori mostrati distinti da quelli reali, nella vista) è arrivata solo dopo aver sbagliato in quel modo.

La seconda è stata l'input in Swing, dove tre difetti diversi compilavano senza un avviso e si vedevano solo giocando: una `InputMap` tiene una sola azione per combinazione, un rilascio con un modificatore premuto non scatta, e un elenco di direzioni premute non basta se ogni direzione ha due tasti. Nessuno dei tre era un errore di logica: erano tre cose che non sapevamo della libreria.

La terza è stata la più silenziosa: la somma di quindici quindicesimi, in virgola mobile, non fa esattamente 1. Senza una tolleranza nel confronto, ogni passo durava un aggiornamento in più e il gioco andava di un sedicesimo più lento, senza che niente lo segnalasse. È l'errore che ci ha convinti a scrivere prove che contano gli aggiornamenti e non solo prove che guardano il risultato finale.

4.3 Commenti per i docenti

Tre scelte sono deliberate e vorremmo motivarle brevemente, perché potrebbero sembrare omissioni.

- **Nessuna libreria esterna.** Il progetto usa solo Java 21 e Swing, che è nel JDK. Non abbiamo trovato una dipendenza che facesse qualcosa che non potessimo fare meglio a mano in questo dominio, e il vantaggio pratico è che il progetto si compila e si esegue dopo una semplice importazione, su qualunque macchina con il JDK 21.
- **Nessun asset binario fra i sorgenti di logica.** Tutta la grafica sta in tre file PNG dentro `src/risorse`, disegnati da noi; non ci sono materiali di terze parti. I Pokémon, i loro nomi, gli sprite e l'ambientazione sono originali.
- **L'assenza di `default` negli `switch` sulle gerarchie sigillate** è una scelta, non una dimenticanza: è ciò che fa fallire la compilazione quando si aggiunge un caso senza gestirlo.

Infine una nota sul metodo: le affermazioni quantitative in questa relazione (il numero di classi e righe, le 112 prove, le tabelle di bilanciamento, la tabella dei difetti deliberati) sono state misurate sul codice consegnato, non stimate.

Appendice A

Guida utente

A.1 Requisiti

JDK 21 o superiore (il JDK, non il solo runtime: il gioco va compilato) e un ambiente grafico. Nessuna libreria esterna da scaricare.

A.2 Come avviare il gioco

Tre modi, tutti equivalenti.

Da Eclipse. Importare il progetto (File > Import > Existing Projects into Workspace), poi tasto destro su `gdr.grafica.Avvio` > Run As > Java Application. Eclipse copia da sé i fogli sprite accanto ai `.class`.

Su Windows. Doppio clic su `gioca.bat`, nella cartella del progetto, oppure da `cmd`:

```
gioca.bat
```

Su Linux e macOS. Dalla cartella del progetto:

```
./gioca.sh
```

I due script fanno la stessa cosa: compilano i sorgenti in `bin`, copiano `src/risorse` accanto ai `.class` e avviano `gdr.grafica.Avvio`. La copia delle risorse serve perché `javac` copia solo i `.class`: senza quel passaggio i fogli sprite non si trovano nel classpath e il gioco non parte. Se manca `javac`, `gioca.bat` lo dice e si ferma, invece di fallire con un errore oscuro.

Nota. `gioca.bat` genera un file temporaneo `sorgenti.txt` con l'elenco dei sorgenti e lo cancella da solo a compilazione riuscita; se resta, è perché la compilazione è fallita, e si può eliminare a mano.

A.3 Comandi

Tasto	Nell'esplorazione	In battaglia
Frecce direzionali	cammina nelle quattro direzioni	su e giù scorrono il menu
W A S D	come le frecce	W S scorrono il menu
Invio, Spazio o Z	—	confermano la voce scelta

Tenendo premuta una direzione il personaggio cammina senza fermarsi fra una casella e l'altra. Premendo due direzioni insieme vince quella premuta per ultima.

A.4 Come si gioca

Si cammina sulla mappa esplorando due zone collegate da un passaggio. Le caselle di erba alta (le più scure) possono far comparire un Pokémon selvatico: l'incontro apre una battaglia a turni. Dopo un incontro i due passi successivi sono sicuri, dal terzo un altro incontro può ripartire; i passi contano anche fuori dall'erba, quindi tornando sul sentiero la tregua si consuma comunque.

In battaglia si scelgono una mossa o la fuga. Chi è più veloce agisce per primo; a parità di velocità tocca al giocatore. Il danno dipende dalle statistiche, dall'elemento della mossa contro quello dell'avversario e da una piccola variazione casuale, ma un colpo a segno fa sempre almeno un punto.

Dopo una battaglia il Pokémon conserva i danni: vincere non cura e non fa salire di livello. Se invece il Pokémon esaurisce i punti salute, si viene riportati al punto di partenza con il Pokémon curato, così una battaglia persa non blocca la partita.

L'ospedale — l'edificio con la croce azzurra — cura il Pokémon: basta camminare contro la sua porta. La visita dura qualche secondo, durante i quali i comandi sono ignorati.

A.5 Struttura della consegna

```
progetto/
  src/
    gdr/
      nucleo/      Posizione, Direzione, Casuale, CasualeStandard
      mondo/       TipoTile, Passaggio, Zona, LettoreZona, Mondo
      entita/      MovimentoGriglia, Giocatore
      battaglia/   Elemento, Mossa, Specie, Creatura,
                   CalcolatoreDanno, AzioneBattaglia,
                   EventoBattaglia, Esito, Battaglia
      incontri/    VoceIncontro, TabellaIncontri, GeneratoreIncontri
      partita/     FaseGioco, Partita, EventoPartita,
                   Esplorazione, EventoEsplorazione
      dati/        ZoneIniziali, Bestiario, Repertorio
      grafica/     Avvio, FinestraGioco, PannelloGioco, CicloDiGioco, ...
    risorse/       terreno.png, personaggio.png, creature.png
  test/
    gdr/           le 11 classi *Test, piu' CasualeDiProva e MappeDiProva
  gioca.sh         compila ed avvia (Linux, macOS)
  gioca.bat        compila ed avvia (Windows)
  .project .classpath   progetto Eclipse, con test/ come sorgente di prova
  LEGGIMI.md
```